Adaptive Frame Selection for Efficient Video Understanding

Computer Vision — Course Project Proposal

1. What the Topic Is

Video data is overwhelmingly redundant. Nearly every video model in use today samples frames **uniformly** — take 8, 16 or 32 frames at even spacing and feed them to the network. For a three-minute video at 1 fps there are roughly 180 candidate frames, yet the information actually needed to answer a question typically lives in just two to five of them. Uniform sampling therefore spends most of its compute budget on frames that carry no useful signal.

The problem this project addresses can be stated precisely:

> *Given a video of N frames, a frozen downstream model, and a budget of only K frames (K much smaller than N), which K frames should be selected — and how much accuracy does a task-aware choice recover over uniform sampling at identical compute cost?*

The insight that makes this non-trivial: simply picking the K highest-scoring frames does not work. The top-ranked frames are almost always near-duplicates of one another, drawn from the same two-second window. The genuine objective balances **relevance, diversity and temporal coverage** under a fixed budget. It is a subset-selection problem, not a ranking problem — and that reframing is the technical core of the project.

2. What We Will Build

Setting: question-conditioned frame selection for long-video question answering, with a frozen vision–language model as the reader. This is the current research frontier (2025–26) and remains computationally cheap if the pipeline is engineered correctly.

Pipeline

Stage	Choice	Rationale
Decode	1 fps to N candidate frames	180-900 candidates per video
Cheap encoder	CLIP / SigLIP ViT-B, features cached to disk	One-time cost; every later experiment then runs in seconds
Selector (our contribution)	relevance + diversity + temporal coverage	The actual research component
Reader	Qwen2.5-VL 3B/7B, frozen, K frames only	Gains attributable solely to selection, not to training
Metrics	accuracy vs. K, vs. GFLOPs, wall-clock latency	Pareto curves, not single numbers

Caching the frame features is the single most important engineering decision. Once features are on disk, a full selection experiment takes seconds rather than hours.

Method

Each frame is scored against the question, and the final subset is chosen by greedy **submodular maximisation** over a facility-location / MMR objective. This is training-free, runs in milliseconds, and carries a known (1 - 1/e) approximation guarantee — which supplies a genuine theoretical result at no additional cost.

Baselines (mandatory)

Uniform, random, fixed-stride, and CLIP top-K without diversity. Critically, we also compute an **oracle**: a search for the best possible K-frame subset using ground-truth answers. The oracle quantifies how much headroom exists at all, and that plot is typically the most persuasive figure in the entire report.

Datasets

NExT-QA (short-form, temporal reasoning) together with an EgoSchema subset or LongVideoBench (long-form). Between 500 and 1000 questions per dataset is sufficient — depth of analysis is rewarded far more than breadth of benchmarks.

Differentiators (we select one or two)

- **Per-video adaptive budget.** Spend 4 frames on easy questions and 32 on hard ones while holding *average* compute fixed. Nearly every published method fixes K; making K itself adaptive is the sharpest reading of our own title and is genuinely underexplored.
- **Distilled selector.** Train a two-layer transformer (under 5M parameters) to imitate the oracle's choices, yielding a learned selector without training any video model.
- **When does adaptive selection actually matter?** Measure how many frames each dataset truly requires, showing that uniform sampling is already near-optimal where a single frame suffices, while adaptive selection wins on long-horizon and needle-style tasks. A well-measured honest negative result scores better than an inflated win.
- **Synthetic needle probe.** Splice a known visual cue into one frame of a long video. Cheap, fully controlled, and proves the mechanism works.

3. Project Skeleton

- **Phase 1 — Setup and data**
  Fix the two datasets and question subsets. Write the decoding and feature-caching script. Nothing else begins until features are cached.
- **Phase 2 — Baselines and oracle**
  Implement uniform, random, stride and CLIP top-K. Run the frozen reader at K = 1, 2, 4, 8, 16, 32. Compute the oracle upper bound. The full performance envelope is then known before a single line of our own method is written.
- **Phase 3 — Our selector**
  Relevance scoring followed by submodular diverse subset selection with a temporal coverage term. Tune the diversity weight.
- **Phase 4 — The differentiator**
  Implement the adaptive budget and / or the distilled selector.
- **Phase 5 — Analysis and ablations**
  Ablate each component. Break results down by question type. Report FLOPs and latency alongside accuracy. Run the synthetic needle probe.
- **Phase 6 — Deliverables**
  Report in CVPR format, a clean reproducible repository with configs and cached features, and a short demo visualising which frames were selected and why. The demo is disproportionately effective in a viva.

4. What Earns the Grade

Rigour, not model size. Concretely: Pareto curves of accuracy against GFLOPs rather than tables of isolated numbers; the oracle upper bound; a proper per-component ablation; a per-category breakdown; wall-clock latency reported next to FLOPs; and a reproducible repository.

Feasibility. The vision–language reader is the only expensive component. Using the 3B variant, fixing the question subsets early and caching every feature keeps the experiment loop at minutes rather than hours, and the entire project runs comfortably on a single lab GPU.